PART A

Program – 1:

Vending machine (VendingMachine.java, main in this file)

(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.

(d) Stop the loop once the total reaches 15 or more.

(e) Print the change to return (total − 15).

Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

Code :

```
import java.util.scanner;
class VendingMachine
{
 enum Coin{ONE,TWO,FIVE,TEN}
 public static void main(String arg[])
 {
 Scanner sc=new Scanner(System.in);
 int p=15;
 int t=0;
 while(t<p)
 {
 System.out.println("Enter the coin");
 Coin c=Coin.valueOf(sc.next().toUpperCase());
 int value=0;
```

```
switch(c)
{
case ONE:
value=1;
break;
case TWO:
value=2;
break;
case FIVE:
value=5;
break;
case TEN:
value=10;
break;
default:
System.out.println("Invalid");  }
t+=value;
}
System.out.println("Total="+t);
System.out.println("Paid Change="+(t-p));  }
}
```

Output :

```
Output
Enter the coin
one
Enter the coin
two
Enter the coin
five
Enter the coin
ten
Total=18
Paid Change=3
roll no. 25AIML005

=== Code Execution Successful ===
```

Program – 2:

(a) Define a record Vehicle(String number, String type).

(b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types "done" for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's counter.

(d) After the loop, print the total toll and the type with the highest count.

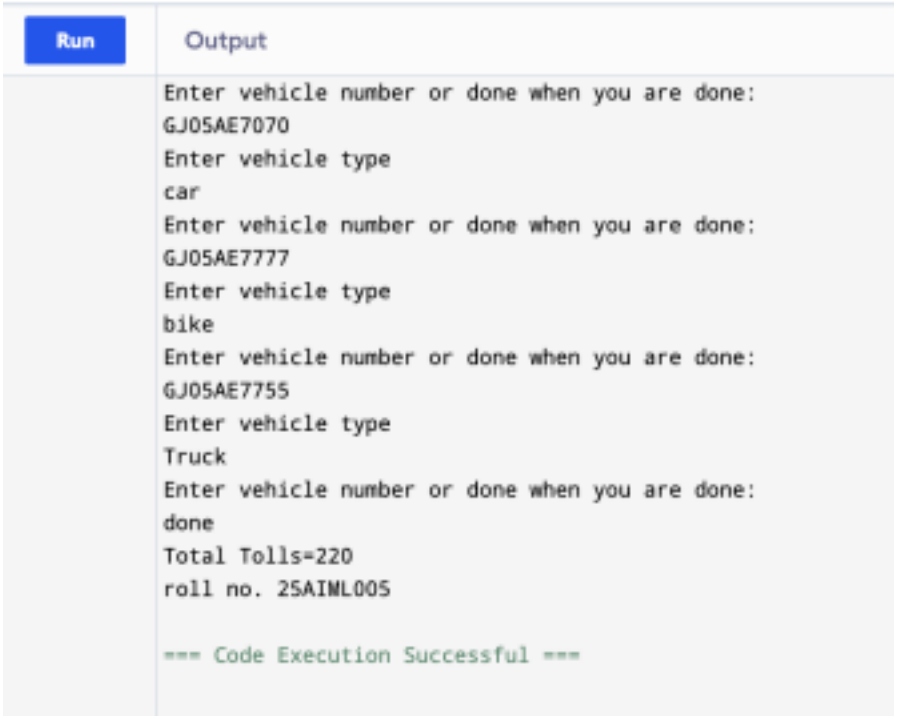
Expected output: two cars + one bike → "Total toll: 120", "Most frequent: car". **Code :**

```
import java.util.*;
class TollBooth
{
 record Vehicle(String number, String type){}
 public static void main(String arg[])
 {
 Scanner sc=new Scanner(System.in);
```

```
int car=0,bike=0,truck=0,t=0;
while(true)
{
System.out.println("Enter vehicle numver or done when you are done:");
String number=sc.next();
if(number.equalsIgnoreCase("done"))
break;
System.out.println("Enter vehicle type");
String type=sc.next().toLowerCase();
Vehicle v=new Vehicle(number,type);
int toll=switch(v.type())
{
case "bike"->20;
case "car"->50;
case "truck"->150;
default->0;
};
t+=toll;
switch(v.type())
{
case "bike"->bike++;
case "car"->car++;
case "truck"->truck++;
}
}
System.out.println("Total Tolls="+t);
if(bike>car && bike>truck)
System.out.println("Most frequent=Bike");
else if(car>bike && car>truck)
```

```
System.out.println("Most frequent=Car");
else if(truck>car && truck>bike)
System.out.println("Most frequent=Truck");
}
}
```

Output :



Program – 3:

(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases).

(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score.

(d) After 5 rounds, print the overall winner.

Expected output: each round prints both moves and the round winner; ends with a "You win 3–2" summary.

Code :

```
import java.util.*;
class RPSLS
{
 enum Move{ROCK,PAPER,SCISSORS,LIZARD,SPOCK}
static int winner(Move a, Move b)
 {
 if(a==b)
 return 0;
 switch (a)
 {
 case ROCK:
 if(b==Move.SCISSORS || b==Move.LIZARD)
 return 1;
 else
 return -1;
 case PAPER:
 if(b==Move.ROCK||b==Move.SPOCK)
 return 1;
 else
 return -1;
 case SCISSORS:
 if(b==Move.PAPER||b==Move.LIZARD)
 return 1;
 else
 return -1;
```

```
 case LIZARD:
 if(b==Move.PAPER||b==Move.SPOCK)  return
1;
 else
 return -1;
 case SPOCK:
 if(b==Move.ROCK||b==Move.SCISSORS)  return
1;
 else
 return -1;
 }
 return 0;
 }

 public static void main(String arg[])
 {
 Scanner sc=new Scanner(System.in);
 Random r=new Random();
 Move moves[]=Move.values();
 int playerScore=0,compScore=0;
 for(int i=1;i<=5;i++)
 {
 System.out.println("Enter your move:");
 Move player=Move.valueOf(sc.next().toUpperCase());  Move
computer=moves[r.nextInt(moves.length)];
System.out.println("Your move:"+player);
System.out.println("Computer move:"+computer);  int
```

```
result=winner(player,computer);
 if(result==1)
 {
```



```
 System.out.println("You won!");
playerScore++;
 }
 else if(result==-1)
 {
 System.out.println("Computer won!");
compScore++;
 }
 else
 System.out.println("It's a tie!");
 }
 System.out.println("\nFinal Score:");
 System.out.println("Your Score:"+playerScore);
System.out.println("Computer's Score:"+compScore);
if(playerScore>compScore)
 System.out.println("You are the final winner!");
else if(playerScore<compScore)
 System.out.println("Computer is the final winner!");
else
 System.out.println("It's a Tie!");
 }
}
```

Output :


ore
Run
Output
Enter your move:
rock
Your move:ROCK
Computer move:LIZARD
You won!
Enter your move:
lizard
Your move:LIZARD
Computer move:PAPER
You won!
Enter your move:
paper
Your move:PAPER
Computer move:LIZARD
Computer won!
Enter your move:
rock
Your move:ROCK
Computer move:SCISSORS
You won!
Final Score:
Your Score:3
Computer's Score:2
You are the final winner!
roll no. 25AIML005
=== Code Execution Successful ===

PRACTICAL-2

PART A:

Program – 1:

Smart thermostat (Thermostat.java, main in this file)

(a) Create class Thermostat with private String location, private int temperature, private
static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is
within MIN..MAX else 22; increment activeCount.
(c) Constructor(location): chain with this(location, 22).
(d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)".
(e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)".
(f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). Expected output: temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.

CODE:

```
public class Pract2_1 {
   static class Thermostat {
      private String location;
      private int temp;
      private static int Min = 16;
      private static int Max = 30;
      private static int activeCount=0;

      Thermostat(String location , int startTemp){
         this.location = location;
         if(startTemp >= Min && startTemp <= Max){
```

```
            this.temp = startTemp;
        }else{
            this.temp = 22;
        }
        activeCount++;
    }

    Thermostat(String location){
        this.location = location;
        this.temp = 22;
        activeCount++;
    }

    public void raise() {
        if (temp < Max) {
            temp++;
        } else {
            System.out.println("Already at maximum : " +
        Max); }
    }

    public void lower() {
        if (temp > Min) {
            temp--;
        } else {
            System.out.println("Already at minimum : "+ Min);
        }
    }

    public void getTemp(){
        System.out.println(temp);
    }

    public void getCount(){
        System.out.println(activeCount);
    }
}

public static void main(String args[]){

    Thermostat t1 = new
    Thermostat("Charusat",26); Thermostat t2 =
```

```
        new Thermostat("Charusat");
```

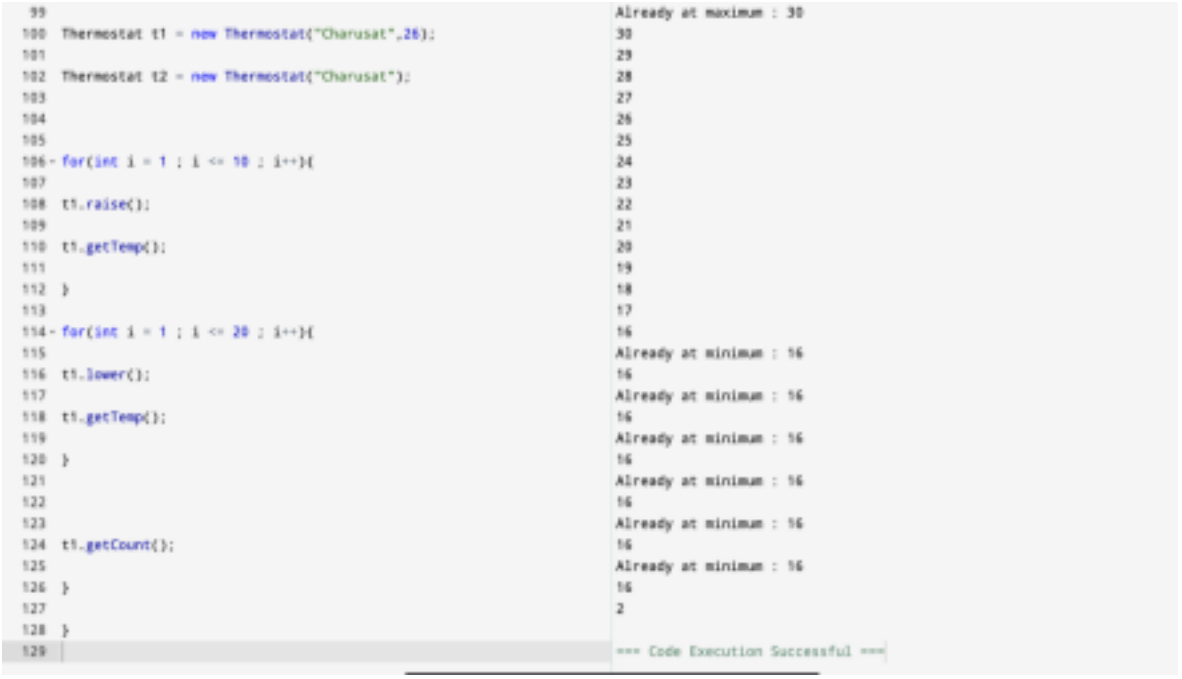


```
for(int i = 1 ; i <= 10 ; i++){
t1.raise();
t1.getTemp();
}
for(int i = 1 ; i <= 20 ; i++){
t1.lower();
t1.getTemp();
}

t1.getCount();
}
}
```

OUTPUT:





```
99
100 Thermostat t1 = new Thermostat("Charusat",26);
101
102 Thermostat t2 = new Thermostat("Charusat");
103
104
105
106 - for(int i = 1 ; i <= 10 ; i++){
107
108 t1.raise();
109
110 t1.getTemp();
111
112 }
113
114 - for(int i = 1 ; i <= 20 ; i++){
115
116 t1.lower();
117
118 t1.getTemp();
119
120 }
121
122
123
124 t1.getCount();
125
126 }
127
128 }
129
```

```
Already at maximum : 30
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
2

=== Code Execution Successful ===
```

Program – 2:
Cinema show (CinemaShow.java, main in this file)
(a) Create class CinemaShow with private String title, private int seatsAvailable, private
final int capacity, and private static int totalBooked=0.
(b) Constructor(title, capacity): set both and seatsAvailable = capacity.
Constructor(title): chain with this(title, 100).
(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked,
return true; else return false unchanged.
(d) cancel(int n): increase seatsAvailable by n but never above capacity.
(e) Add getSeatsAvailable() and static getTotalBooked().
(f) In main: run a sequence of book and cancel calls, printing the result and seats after
each, then print getTotalBooked().
Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

Code :

```
public class Pract2_2 {
   public static class CinemaShow{
      private String title;
      private int seatAvailable;
      private static int totalBooked = 0;
      private final int capacity = 200;

      CinemaShow(String title , int capacity){
         this.title = title;
         this.seatAvailable = capacity;
      }
      CinemaShow(String title){
         this(title,100);
      }

      public boolean book(int n){
         if(n <= seatAvailable){
            seatAvailable -= n;
            totalBooked += n;
            return true;
          }
         return false;
      }
```

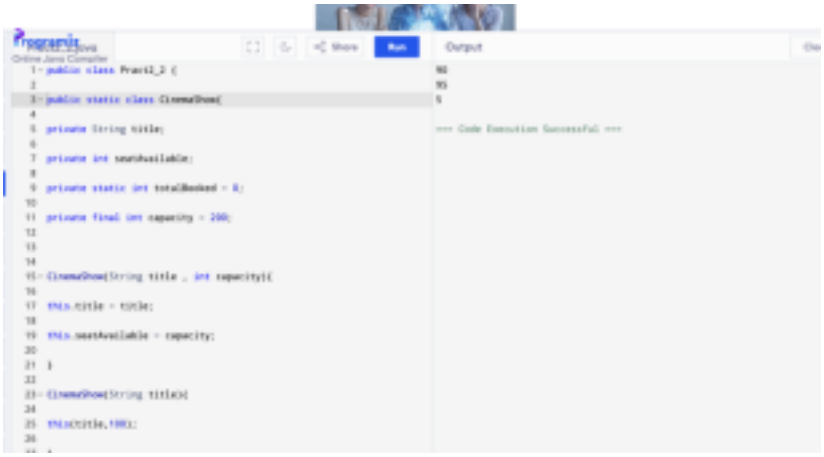
```
    public void cancle(int seat){
        int a = seatAvailable + seat;
        if(a <= capacity && a >= 0){
            seatAvailable += seat;
            totalBooked -= seat;
        }

    }
}

  public static void main(String[] args){
        CinemaShow c = new CinemaShow("Spyder Man : Brand New
Day",100);

    c.book(10);
    System.out.println(c.seatAvailable);
    c.cancle(5);
    System.out.println(c.seatAvailable);
    System.out.println(c.totalBooked);
    }
}
```

Output :



Program – 3:

(For Advanced Learners – home) Parking lot (ParkingLot.java) (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.
(b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to
revenue; for "four", if fourWheelers < fourCap increment and add 40; otherwise print
"Full" and reject.
(c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print
final occupancy and revenue.
Expected output: parking into a full section is refused; revenue = 20×(two parked)
+ 40×(four parked).

Code :

```
class Parkinglot {sss

   private int twoWheelers = 0;
   private int fourWheelers = 0;

   private final int twoCap;
   private final int fourCap;

   private static long revenue = 0;

   Parkinglot(int twoCap, int fourCap) {
      this.twoCap = twoCap;
      this.fourCap = fourCap;
   }

   public void park(String type) {
      if (type.equals("two")) {
```

```
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                System.out.println("Two-wheeler parked");
            } else {
                System.out.println("Two-wheeler section Full");
```



```
            }
        } else if (type.equals("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                System.out.println("Four wheeler
            parked"); } else {
                System.out.println("Four wheeler section
            Full"); }
        }
    }

    public void leave(String type) {
        if (type.equals("two")) {
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two wheeler left");
            }
        } else if (type.equals("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four wheeler left");
            }
        }
    }

    public void display() {
        System.out.println("Two-wheelers: " + twoWheelers);
    System.out.println("Four-wheelers: " + fourWheelers);
    System.out.println("Revenue: " + revenue); }

    public static void main(String[] args) {

        Parkinglot p = new Parkinglot(2, 2);
```

```
p.park("two");
p.park("two");
p.park("two");

p.park("four");
p.park("four");
```

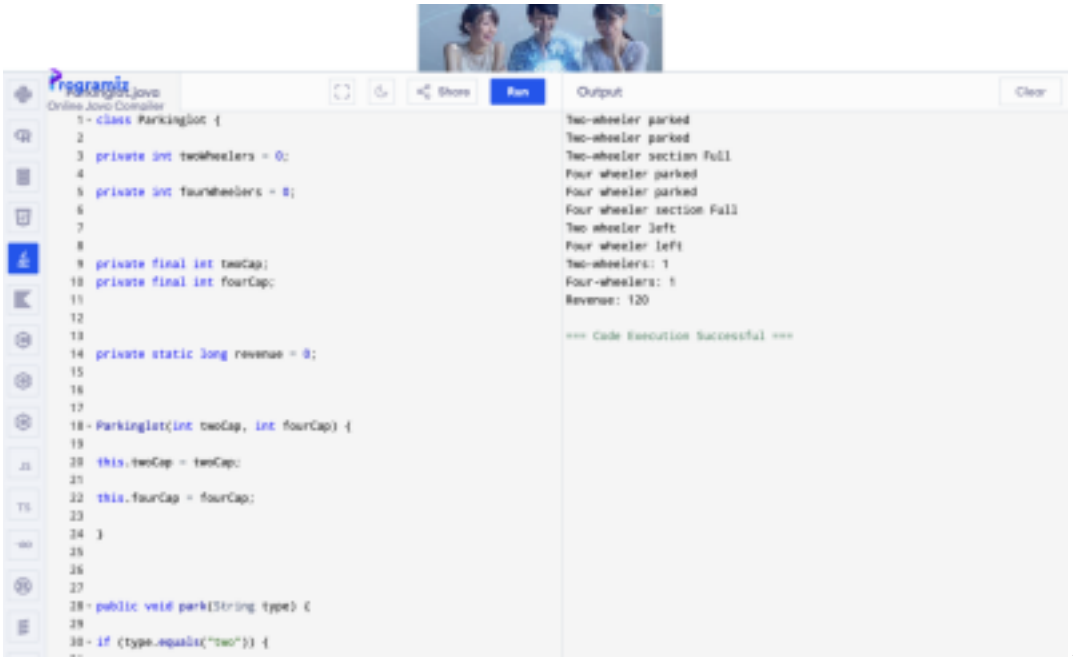


```
p.park("four");

p.leave("two");
p.leave("four");

p.display();
}
}
```

Output :



PART B:

Part B — Project Work (Hour 2): MiniBank

In Practical 1 you built the empty menu shell. Now you create the data the bank works on —
the Customer and Account classes. Using encapsulation, an account's balance

can change
only through deposit() and withdraw(), never directly. These two classes are the foundation
every later lab extends.
1. Create a class named Customer with private fields name, email and mobile (all String)
and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as
“CUST101”. Add a constructor and public getter methods.
2. Create a class named Account with private fields: a final String accountNumber, a String
ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example “AC0001”). 3. Add two constructors to Account: one taking ownerName and an opening balance; one
taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public
method withdraw(long amount) that subtracts amount and returns true when the balance
is sufficient, otherwise returns false without changing the balance. Add getter methods.
Do NOT add a public setter for balance.
5. In main, create three Account objects inside an Account[] array, perform a few deposits
and withdrawals, and print each balance.

CODE:


```
class Customer {
```

```
private String name;
private String email;
private String mobile;
private final String customerId;

private static long customerCounter = 101;

private static String generateCustomerId() {
   return "CUST" + customerCounter++;
}

public Customer(String name, String email, String mobile)
   { this.name = name;
   this.email = email;
   this.mobile = mobile;
   this.customerId = generateCustomerId();
}

public String getName() {
   return name;
}
```

```
    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId()
        { return customerId;
    }
}

class Account {

    private final String
    accountNumber; private String
    ownerName;
    private long balance;
    private boolean active;

    private static int counter = 1;
```

```
private static String generateAccountNumber()
{ return String.format("AC%04d", counter++);
}

public Account(String ownerName, long balance) {
   this.accountNumber =
   generateAccountNumber(); this.ownerName =
   ownerName;
   this.balance = balance;
   this.active = true;
}

public Account(String ownerName) {
   this(ownerName, 0);
}

public void deposit(long amount) {
   if (amount > 0)
      balance += amount;
}
```

```
public boolean withdraw(long amount)
    { if (amount <= balance) {
        balance -= amount;
```



```
        return true;
    }
   return false;
}

public String getAccountNumber()
    { return accountNumber;
}

public String getOwnerName()
    { return ownerName;
}

public long getBalance() {
   return balance;
}

public boolean isActive() {
```

```
        return active;
    }
}

public class MiniBank {
```



```
    public static void main(String[] args) {

        Customer c1 = new Customer("Daksh",
"daksh18@gmail.com",  "9909900018");
        Customer c2 = new Customer("Kartavya",
"kartavya31@gmail.com", "9909900031");
        Customer c3 = new Customer("meet",
"meet13@gmail.com",  "9909900013");

        System.out.println(c1.getCustomerId()  +  "  "  +
        c1.getName()); System.out.println(c2.getCustomerId() + " "
        + c2.getName()); System.out.println(c3.getCustomerId() + "
        " + c3.getName());

        Account[] accounts = new Account[3];
```

```
        accounts[0] = new Account("Daksh", 500000);
        accounts[1] = new Account("Kartavya");
        accounts[2] = new Account("meet", 350000);

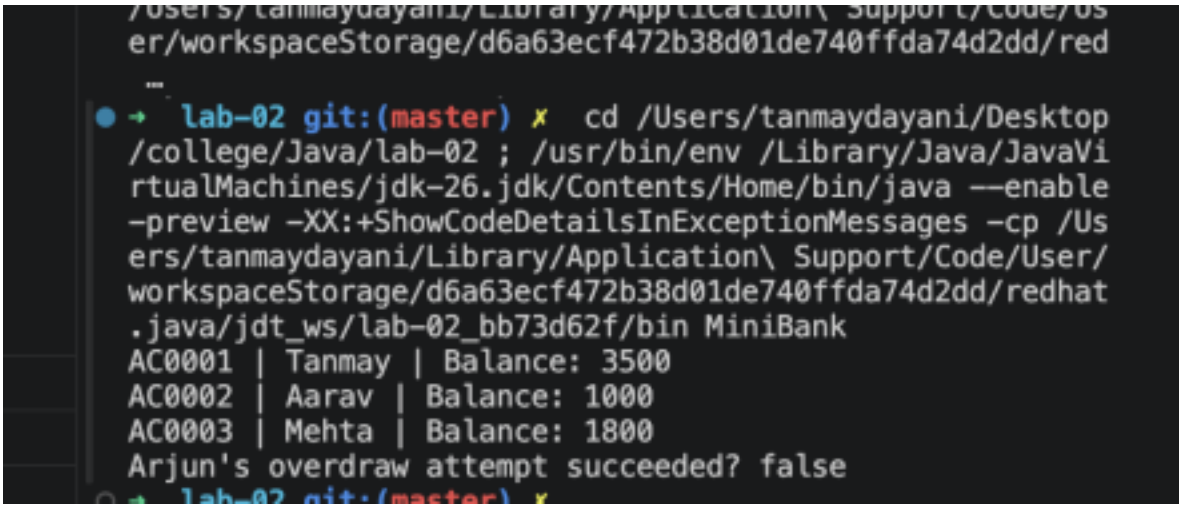
        accounts[0].deposit(100000);
        accounts[1].deposit(250000);
        accounts[2].withdraw(150000);
```



```
        accounts[0].withdraw(1700000);

        for (Account acc : accounts) {
            System.out.println("Account No : " +
 acc.getAccountNumber());
            System.out.println("Owner : " + acc.getOwnerName());
            System.out.println("Balance : " + acc.getBalance());
            System.out.println();
        }
    }
}
```

OUTPUT:



GITHUB LINK: https://github.com/tanmay-dayani/Java.git

PROBLEM:- 3.1	Distinct points (Point.java + Driver.java with main) (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return “(x, y)”. (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print “Distinct: N”.

CODE :-

```
import java.util.Objects;

public class Practical3_1 {
 static class Point {
 private int x;
 private int y;

 Point(int x , int y){
 this.x = x;
 this.y = y;
 }

 @Override
 public String toString() {
 return "(" + this.x + ", " + this.y + ")";
 }

 @Override
 public boolean equals(Object obj) {
 if (this == obj) {
 return true;
 }
 if (obj == null || getClass() != obj.getClass()) {  return
false;
 }
 Point other = (Point) obj;
 return this.x == other.x && this.y == other.y;  }

 @Override
 public int hashCode() {
 return Objects.hash(this.x, this.y);
 }
 }

 public static void main(String args[]){
 Point p[] = {
 new Point(1,2),
 new Point(2,3),
 new Point(4,5),
 new Point(1,3),
 new Point(6,7),
 new Point(8,9),
 new Point(2,7),
 new Point(4,3),
 };
```

```
int count = 0;

 for(int i = 0 ; i < p.length ; i++){
 boolean check = false;

 for(int j = 0 ; j < i ; j++){
 if(p[i].equals(p[j])){
 check = true;
 break;
 }
 }

 if(!check){
 count++;
 }
 }

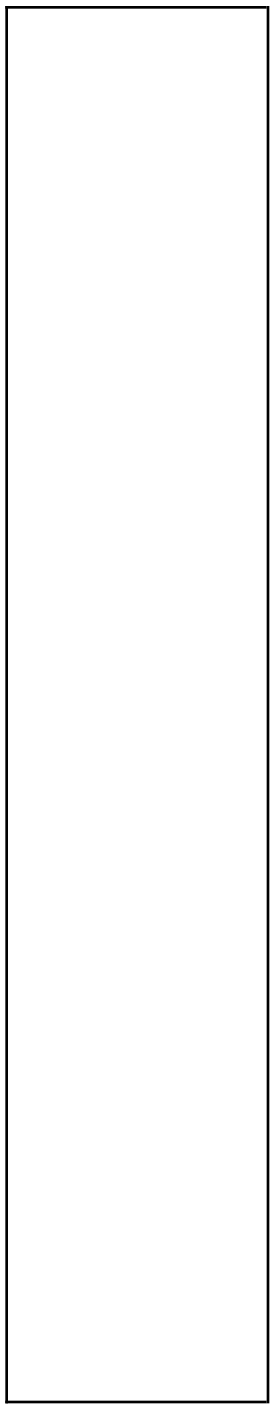
 System.out.println(count);
 }
}
```


OUTPUT :-



Problame_3.2	Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: " the first time a repeat appears.

Code:-	`import java.util.*;`
	`import java.util.Objects;`
	
	`public class Practical3_2 {`
	`static class Card {`
	`private String rank;`
	`private String suit;`
	
	`Card(String rank, String suit) {`
	`this.rank = rank;`
	`this.suit = suit;`
	`}`
	
	`@Override`
	`public String toString() {`
	`return "(" + this.rank + ", " + this.suit + ")";`
	`}`
	
	`@Override`
	`public boolean equals(Object obj) {`
	`if (this == obj) {`
	`return true;`
	`}`
	`if (obj == null || getClass() != obj.getClass()) {`
	`return false;`
	`}`
	`Card other = (Card) obj;`



```
return Objects.equals(this.rank, other.rank) && Objects.equals(this.suit, other.suit);
}

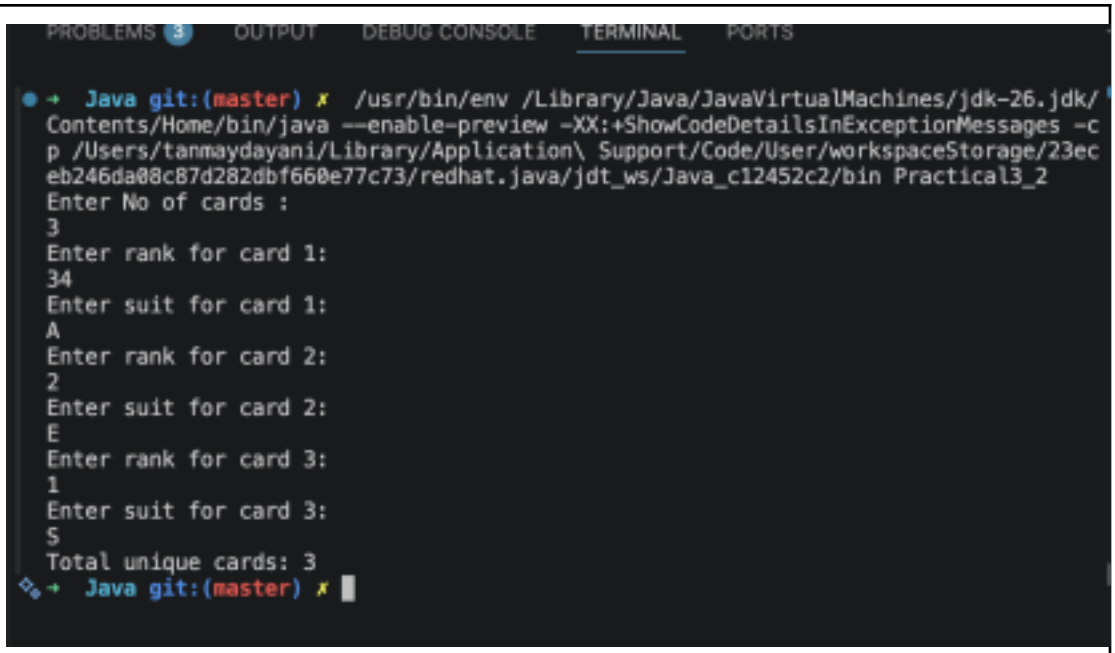
 @Override
public int hashCode() {
return Objects.hash(this.rank, this.suit);
}
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.println("Enter No of cards : ");
int n = sc.nextInt();
sc.nextLine();
```

```
Card c[] = new Card[n];
int count = 0;

for (int i = 0; i < n; i++) {
boolean check = false;
System.out.println("Enter rank for card " + (i + 1) + ": ");
String r = sc.nextLine();
System.out.println("Enter suit for card " + (i + 1) + ": ");
String s = sc.nextLine();
c[i] = new Card(r, s);

for (int j = 0; j < i; j++) {
if (c[i].equals(c[j])) {
check = true;
break;
}
}
if (!check) {
count++;
}
}
System.out.println("Total unique cards: " + count);
sc.close();
}
}
```

OUTPUT:-	
Problame_3.3	(a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g. (c) Override toString() to return num + “/” + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

CODE:-

```
public class Practicle3_3 {
public static void main(String[] args) {
Fraction first = new Fraction(1, 2);
Fraction second = new Fraction(2, 4);
Fraction third = new Fraction(3, 6);

System.out.println(first);
System.out.println(second);
System.out.println(third);
System.out.println("All equal: "
+ (first.equals(second) && second.equals(third) && first.equals(third)));
}
}

/** An immutable fraction stored in lowest terms. */
class Fraction {
private int num;
private int den;

public Fraction(int num, int den) {
if (den == 0) {
throw new IllegalArgumentException("Denominator cannot be zero.");
}

long numerator = num;
long denominator = den;
long g = gcd(numerator, denominator);
numerator /= g;
denominator /= g;

// Keep the denominator positive, so equal values have one representation.
if (denominator < 0) {
numerator = -numerator;
denominator = -denominator;
}

this.num = Math.toIntExact(numerator);
this.den = Math.toIntExact(denominator);
}

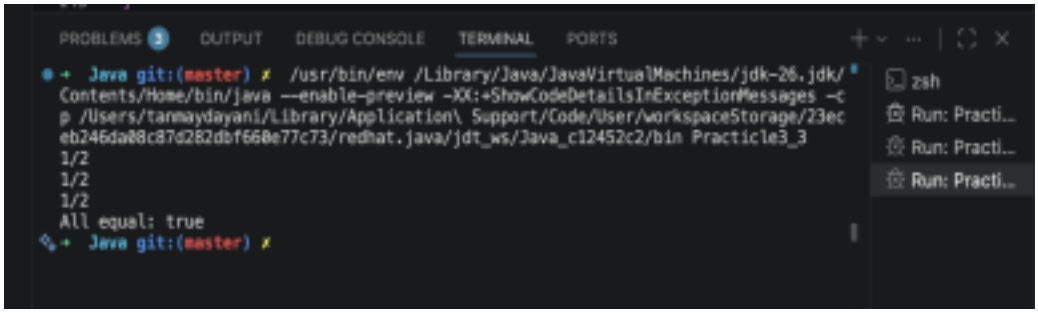
private static long gcd(long a, long b) {
a = Math.abs(a);
b = Math.abs(b);
while (b != 0) {
long remainder = a % b;
a = b;
b = remainder;
}
return a;
}

@Override
public String toString() {
return num + "/" + den;
}
```

```
@Override
public boolean equals(Object other) {
if (this == other) {
return true;
}
if (!(other instanceof Fraction)) {
return false;
}
Fraction fraction = (Fraction) other;
return num == fraction.num && den == fraction.den;
}

@Override
public int hashCode() {
return 31 * num + den;
}
}
```

OUTPUT



MiniProject :-

```
class Customer {

private String name;
private String email;
private String mobile;
private final String customerId;

private static long customerCounter = 101;

private static String generateCustomerId() {
return "CUST" + customerCounter++;
}

public Customer(String name, String email, String mobile) {
```

```
this.name = name;
this.email = email;
this.mobile = mobile;
this.customerId = generateCustomerId();
}

public String getName() {
return name;
}

public String getEmail() {
return email;
}

public String getMobile() {
return mobile;
}

public String getCustomerId() {
return customerId;
}
}

class Account {

private final String accountNumber;
private String ownerName;
private long balance;
private boolean active;

private static int counter = 1;

private static String generateAccountNumber() {
return String.format("AC%04d", counter++);
}

public Account(String ownerName, long balance) {
this.accountNumber = generateAccountNumber();
this.ownerName = ownerName;
this.balance = balance;
this.active = true;
}

public Account(String ownerName) {
this(ownerName, 0);
}

public void deposit(long amount) {
if (amount > 0)
balance += amount;
}

public boolean withdraw(long amount) {
if (amount <= balance) {
```

```
balance -= amount;
return true;
}
return false;
}

public String getAccountNumber() {
return accountNumber;
}

public String getOwnerName() {
return ownerName;
}

public long getBalance() {
return balance;
}

public boolean isActive() {
return active;
}
}

public class Mini_Project {

public static void main(String[] args) {

Customer c1 = new Customer("Tanmay", "tanmay05@gmail.com",
"9909900018"); Customer c2 = new Customer("Kartavya",
"kartavya31@gmail.com", "9909900031"); Customer c3 = new Customer("meet",
"meet13@gmail.com", "9909900013");

System.out.println(c1.getCustomerId() + " " + c1.getName());
System.out.println(c2.getCustomerId() + " " + c2.getName());
System.out.println(c3.getCustomerId() + " " + c3.getName());

Account[] accounts = new Account[3];

accounts[0] = new Account("Tanmay", 400000);
accounts[1] = new Account("kartavya");
accounts[2] = new Account("meet", 950000);

accounts[0].deposit(100000);
accounts[1].deposit(250000);
accounts[2].withdraw(150000);
accounts[0].withdraw(1700000);

for (Account acc : accounts) {
System.out.println("Account No : " + acc.getAccountNumber());
System.out.println("Owner : " + acc.getOwnerName());
System.out.println("Balance : " + acc.getBalance());
System.out.println();
}
}
}
```

Output :-	

GitHub link :- https://github.com/tanmay-dayani/Java.git

Subject:	CEUE203 : OOP-JAVA
Name:	Tanmay Dayani
ID No.:	25AIML005

Practical-4

GIT REPOSITRY LINK : https://github.com/tanmay-dayani/Java.git

Part A

PROBLEM-1 : Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0– 1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.
Expected output: "abc" → Weak; "Abcd1234!" → Strong.

CODE : PasswordChecker.java

```
public class PasswordChecker {

public static boolean hasMinimumLength(String pw) {
return pw.length() >= 8;
}

public static boolean hasUppercase(String pw) {
return pw.matches(".*[A-Z].*");
}
```

```
public static boolean hasDigit(String pw) {
return pw.matches(".*[0-9].*");
}
public static boolean hasSpecialCharacter(String pw) {
return pw.matches(".*[^a-zA-Z0-9].*");
}

public static String strength(String pw) {
int passedRules = 0;

if (hasMinimumLength(pw)) {
passedRules++;
}

if (hasUppercase(pw)) {
passedRules++;
}

if (hasDigit(pw)) {
passedRules++;
}

if (hasSpecialCharacter(pw)) {
passedRules++;
}

if (passedRules <= 1) {
return "Weak";
} else if (passedRules <= 3) {
return "Medium";
} else {
return "Strong";
}
}
}
```

CODE : Driver.java

```
public class Driver {

public static void main(String[] args) {

String[] passwords = {
"abc",
"abcdefgh",
"Abcdefgh",
"Abcd1234",
"Abcd1234!"
};

for (String password : passwords) {
```

```
System.out.println("----------------------------")
; System.out.println("Password: " +
password);
System.out.println("Length >= 8: "
+ PasswordChecker.hasMinimumLength(password));

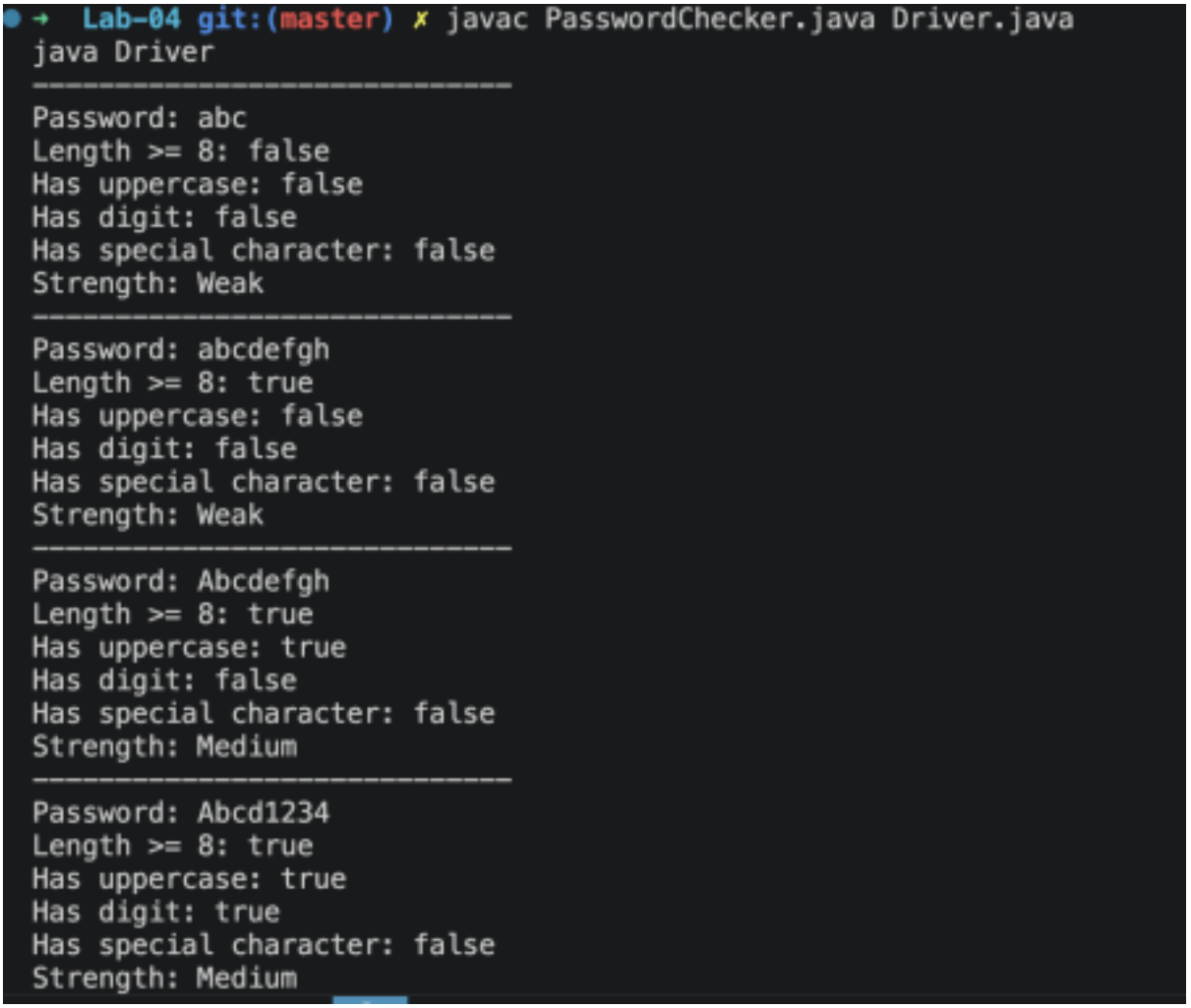
System.out.println("Has uppercase: "
+ PasswordChecker.hasUppercase(password));

System.out.println("Has digit: "
+ PasswordChecker.hasDigit(password));

System.out.println("Has special character: "
+ PasswordChecker.hasSpecialCharacter(password));

System.out.println("Strength: " + PasswordChecker.strength(password));
}
}
}
```

OUTPUT :



PROBLEM-2 : Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).
(b) Read a keyword from the user.
(c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.
(d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.
(e) Print “Matches: N” and the built report.
Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.

CODE : ChatFilter.java

```
public class ChatFilter {
```

```
 public static boolean containsKeyword(String message, String keyword) {
return message.toLowerCase().contains(keyword.toLowerCase());  }
}
```

CODE : Driver.java

```
import java.util.Scanner;

public class Driver {

 public static void main(String[] args) {

 String[] logs = {
 "10:05 alice Hello there",
 "10:10 bob Java is fun",
 "10:15 charlie hello again",
 "badline"
 };

 Scanner input = new Scanner(System.in);

 System.out.print("Enter keyword: ");
 String keyword = input.nextLine();
```

```
int matches = 0;

StringBuilder report = new StringBuilder();


for (String line : logs) {

String[] parts = line.split(" ", 3);


if (parts.length < 3) {

continue;

}


String time = parts[0];

String user = parts[1];

String message = parts[2];


if (ChatFilter.containsKeyword(message, keyword)) {
matches++;

report.append(time)

.append(" ")

.append(user)

.append(": ")

.append(message)

.append("\n");

}

}
```

```
System.out.println("Matches: " + matches);

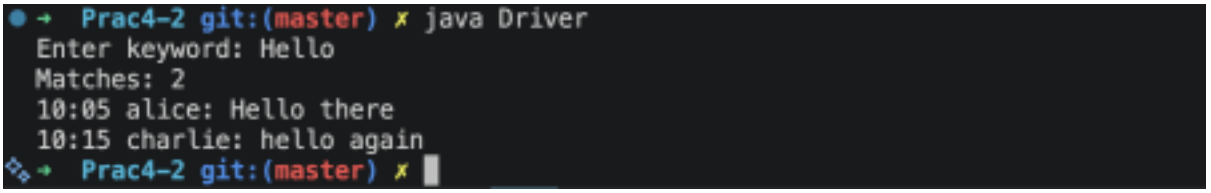
System.out.print(report.toString());


input.close();

}

}
```

OUTPUT :



PROBLEM-3 : Template filler (TemplateFiller.java + Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date}
is not supplied).
(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found. (d) Print the filled template.
Expected output: the unsupplied {date} becomes [?].

CODE : TemplateFiller.java

```
public class TemplateFiller {
```

```
 public static String findValue(String placeholder, String[] names, String[]
values) {
 for (int i = 0; i < names.length; i++) {
 if (names[i].equals(placeholder)) {
 return values[i];
 }
 }

 return "[?]";
 }
}
```

CODE : Driver.java

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Driver {
 public static void main(String[] args) {

 String template = "Dear {name}, order {id} ships {date}.";

 String[] names = {"name", "id"};
 String[] values = {"Riya", "A07"};
```

```
Pattern pattern = Pattern.compile("\\{(\\w+)\\}");

Matcher matcher = pattern.matcher(template);


StringBuilder result = new StringBuilder();


while (matcher.find()) {

String placeholder = matcher.group(1);

String replacement = TemplateFiller.findValue(placeholder, names,
values);


matcher.appendReplacement(result, replacement);  }

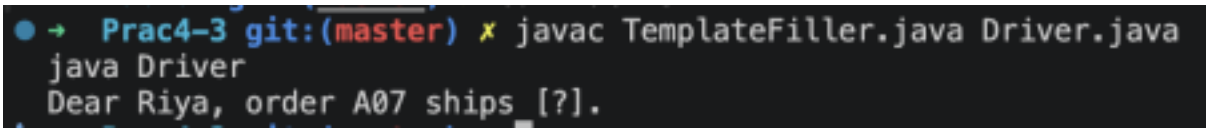

matcher.appendTail(result);


System.out.println(result.toString());

}

}
```

OUTPUT :



Key Questions :

1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?

Answer:

- String: Immutable (cannot be changed).
- StringBuilder: Mutable and faster, but not thread-safe.
- StringBuffer: Mutable and thread-safe, but slower than StringBuilder. - Use String for fixed text, StringBuilder for frequent modifications, and StringBuffer in multi-threaded environments.

2. What does the regular expression [6-9][0-9]{9} match, and why use anchors?

Answer:

- Matches a 10-digit number starting with 6, 7, 8, or 9.
- Anchors (^ and $) ensure the entire input matches the pattern, e.g., ^[6-9][0-9]{9}$.

3. Why should input be validated at the boundary before it is

used? Answer:

- Prevents invalid or malicious data from entering the program. - Improves security, reliability, and reduces errors later in processing.

PRACTICLE : 5

1

AIM:

The aim of this program is to understand abstraction and polymorphism in Java.
It demonstrates how to create an abstract Shape class with an abstract area() method.
Different subclasses such as Circle, Rectangle, and Triangle calculate their areas differently.A Shape[] array is used to store different types of shapes together.
The program uses a single loop to calculate and display each shape's area.
It also calculates the running total and identifies the shape with the largest area.
This program helps understand runtime polymorphism through method overriding.

Code:

```
abstract class Shape {
   abstract double area();
}

class Circle extends Shape {
   double radius;

   Circle(double radius) {
      this.radius = radius;
   }

   @Override
   double area() {
      return Math.PI * radius * radius;
   }
}

class Rectangle extends Shape {
   double length, width;
```

```
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    double area() {
        return length * width;
    }
}

class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    double area() {
        return 0.5 * base * height;
    }
}

public class ShapeDemo {
    public static void main(String[] args) {

        Shape[] shapes = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(3, 8),
            new Circle(2)
```

```
    };

    double total = 0;
    double largest = 0;


    for (Shape s : shapes) {
      double a = s.area();

      System.out.println("Area = " + a);

      total += a;

      if (a > largest) {
        largest = a;
      }
    }

    System.out.println("\n\nTotal area = " + total);
    System.out.println("Largest area = " + largest + "\n");
  }
}
```

Output:

```
➜  Lab-05 git:(master) ✗ java ShapeDemo
Area = 78.53981633974483
Area = 24.0
Area = 12.0
Area = 12.566370614359172


Total area = 127.106186954104
Largest area = 78.53981633974483

➜  Lab-05 git:(master) ✗
```

2

AIM:

to apply abstraction, inheritance, and polymorphism to media items.
It demonstrates an abstract Media class with a common title and late days.
Different media types such as books, DVDs, and magazines calculate late fees differently.
Each subclass overrides the lateFee() method according to its own fee rule.
A mixed Media[] array is used to store different types of media.
The program calculates and displays the late fee for each returned media item.
It also calculates the total late fees for the complete batch.

Code:

```
abstract class Employee {
   String name;
   int id;

   Employee(String name, int id) {
      this.name = name;
      this.id = id;
   }

   abstract double monthlySalary();
}

class FullTime extends Employee {
   double salary;

   FullTime(String name, int id, double salary) {
      super(name, id);
      this.salary = salary;
   }
```

```
    @Override
    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    double hours;
    double rate;

    PartTime(String name, int id, double hours, double rate) {
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }

    @Override
    double monthlySalary() {
        return hours * rate;
    }
}

class Intern extends Employee {
    double stipend;

    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }

    @Override
    double monthlySalary() {
        return stipend;
```

```
    }
}

public class PayrollDemo {
    public static void main(String[] args) {

        Employee[] employees = {
            new FullTime("Rahul", 101, 50000),
            new PartTime("Amit", 102, 80, 300),
            new Intern("Neha", 103, 10000),
            new FullTime("Priya", 104, 60000)
        };

        double total = 0;

        for (Employee e : employees) {

            double salary = e.monthlySalary();

            System.out.println(
                "\n" + e.name + " (ID: " + e.id + ") Salary = " + salary
            );



            if (e instanceof Intern) {
                System.out.println("Note: This employee is an intern.\n");
            }

            total += salary;
        }

        System.out.println("\n\tTotal payroll = " + total + "\n");
    }
}
```

Output:

```
➜  Pr-2 git:(master) ✗ javac PayrollDemo.java
➜  Pr-2 git:(master) ✗ java PayrollDemo

 Ayush (ID: 101) Salary = 5000.0

 Ramesh (ID: 102) Salary = 20000.0

 Suresh (ID: 103) Salary = 110000.0
 Note: This employee is an intern.


 Shreya (ID: 104) Salary = 78000.0

         Total payroll = 213000.0

➜  Pr-2 git:(master) ✗
```

3

AIM:

To develop a Java program to fill placeholders in a given text template.
The program uses a regular expression to identify placeholders in the template.
Each placeholder is matched with its corresponding name in a parallel array.
The associated value is substituted for every supplied placeholder.

If a placeholder has no corresponding value, it is replaced with [?].
A StringBuilder is used to construct the final filled template efficiently.
The completed template is then displayed as the output.

Code:

```
abstract class Media {
   String title;
   int lateDays;

   Media(String title, int lateDays) {
      this.title = title;
      this.lateDays = lateDays;
   }

   abstract double lateFee();
}

class Book extends Media {

   Book(String title, int lateDays) {
      super(title, lateDays);
   }

   @Override
   double lateFee() {
      return lateDays * 2;
   }
}

class DVD extends Media {

   DVD(String title, int lateDays) {
```

```
        super(title, lateDays);
    }

    @Override
    double lateFee() {
        return lateDays * 5;
    }
}

class Magazine extends Media {

    Magazine(String title, int lateDays) {
        super(title, lateDays);
    }

    @Override
    double lateFee() {
        return lateDays * 1;
    }
}

public class MediaDemo {
    public static void main(String[] args) {

        Media[] items = {
            new Book("Java Basics", 3),
            new DVD("Avengers", 2),
            new Magazine("Tech Today", 5)
        };

        double total = 0;

        for (Media m : items) {
```

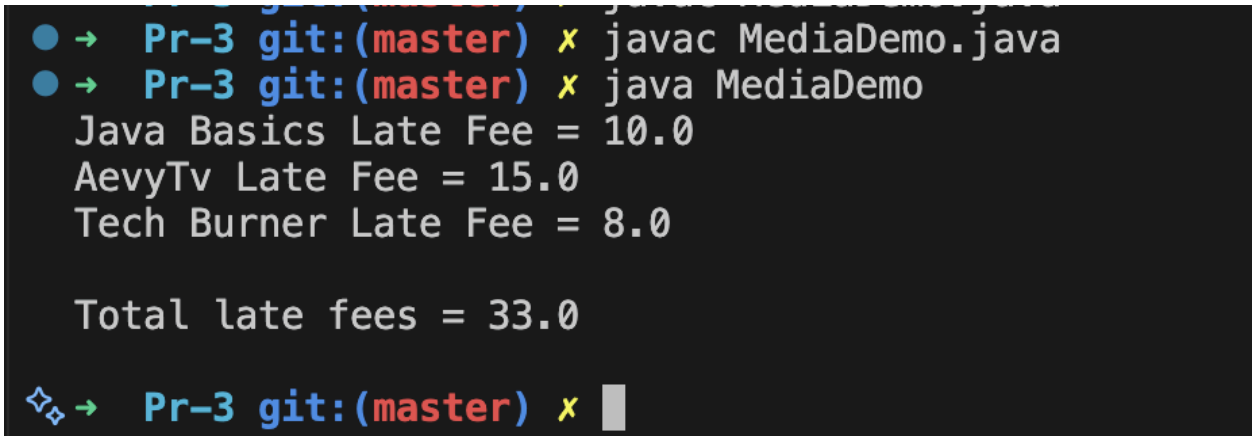
```
        double fee = m.lateFee();

        System.out.println(
           m.title + " Late Fee = " + fee
        );

        total += fee;
      }

      System.out.println("\nTotal late fees = " + total + "\n");
   }
}
```

Output:



Key Questions :

1. Difference between an abstract class and a concrete class?

An **abstract class** is a class that cannot be directly instantiated. It can contain abstract methods that must be implemented by its subclasses. A **concrete class** can be directly instantiated and provides implementations for its methods. For example, Shape can be abstract, while Circle can be a concrete class.

2. What is dynamic method dispatch (run-time polymorphism)?

Dynamic method dispatch is the process where Java decides at **run time** which overridden method to execute. It allows a parent class reference to refer to a child class object. For example, when a Shape reference contains a Circle object, calling area() executes the Circle version of area().

3. What does the super keyword do in a subclass constructor?

The **super** keyword is used to call the constructor of the parent class. It is used to initialize the common properties inherited from the parent class. For example, super(name, id) calls the Employee constructor from FullTime, PartTime, or Intern.

Git Link- https://github.com/tanmay-dayani/Java.git

PRACTICAL – 5

AIM :

1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

CODE SCREENSHOT :

```
import java.util.*;

interface Switchable {

    void on();

    void off();

    default void toggle() {
        System.out.println("Toggling device");
    }
}

class Fan implements Switchable {

    public void on() {
        System.out.println("Fan is ON");
    }

    public void off() {
        System.out.println("Fan is OFF");
    }
}

class Light implements Switchable {

    public void on() {
        System.out.println("Light is ON");
    }
```

```
    public void off() {
        System.out.println("Light is OFF");
    }
}


interface SwitchPermission {

    boolean maySwitchOn(Switchable device, int hour);
}

public class pract6_1 {

    public static void main(String[] args) {

        Switchable[] devices = {
            new Fan(),
            new Light()
        };


        for (Switchable device : devices) {
            device.on();
            device.off();
            device.toggle();
        }

        SwitchPermission permission1 = new SwitchPermission() {

            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

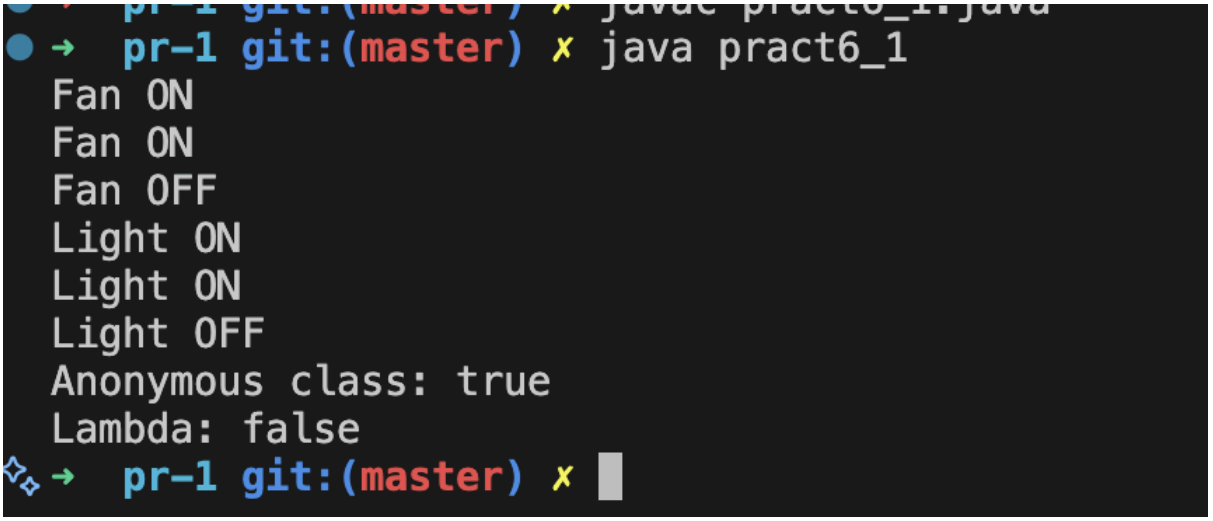
        System.out.println(
            "Anonymous class: " +
            permission1.maySwitchOn(new Fan(), 10)
        );


        SwitchPermission permission2 = (device, hour) -> hour >= 6 && hour <=
22;

        System.out.println(
            "Lambda: " +
```

```
            permission2.maySwitchOn(new Light(), 23)
        );
    }
}
```

OUTPUT SCREENSHOT :



LEARNING OUTCOME :

- Learned to implement abstraction using an abstract Shape class and area() method.
- Learned to use inheritance and method overriding through Circle, Rectangle, and Triangle.
- Applied runtime polymorphism by storing different shape objects in a Shape[] array.
- Learned to use a single loop to calculate individual areas, running total, and the largest area.

2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

CODE SCREENSHOT :

```
interface Notifier {

    void send(String message);

}


interface Urgent {

}

class EmailSender implements Notifier, Urgent {

    public void send(String message) {
        System.out.println("Email: " + message);
    }

}

class SMSSender implements Notifier {

    public void send(String message) {
        System.out.println("SMS: " + message);
    }

}

public class pract6_2 {

    public static void main(String[] args) {

        Notifier email = new EmailSender();
        Notifier sms = new SMSSender();

        Notifier[] senders = {email, sms};
```

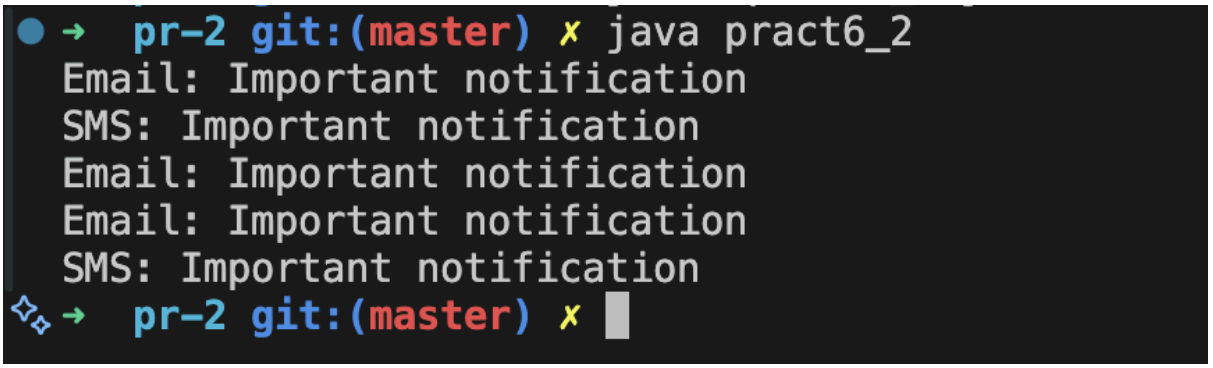
```
        String message = "Your order has been shipped.";

        for (Notifier sender : senders) {

            sender.send(message);


            if (sender instanceof Urgent) {
                sender.send(message);
            }
        }
    }
}
```

OUTPUT SCREENSHOT :



LEARNING OUTCOME :

- Learned to implement an abstract Employee class with an abstract monthlySalary() method.
- Applied inheritance and method overriding for FullTime, PartTime, and Intern.
- Learned to use super() for constructor chaining to initialize common name and ID attributes.
- Applied polymorphism using a mixed Employee[] array to calculate different salaries.
- Learned to use the instanceof operator to identify interns and display a specific note.
- Calculated individual salaries and total payroll using loops.

3. (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

CODE SCREENSHOT :

```
package discount;

@FunctionalInterface
public interface DiscountRule {
    double apply(double price);
}
```

```
package discount;

import java.util.*;

public class DiscountEngine {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        List<Double> prices = Arrays.asList(
            1000.0,
            2500.0,
            5000.0,
            8000.0
        );

        System.out.println("Prices:");

        for (double price : prices) {
            System.out.println(price);
        }

        System.out.println("\nChoose discount rule:");
        System.out.println("1. 10% discount");
        System.out.println("2. 20% discount");
        System.out.println("3. Flat Rs. 500 discount");
```

```
        int choice = sc.nextInt();

        DiscountRule rule;

        if (choice == 1) {

            rule = price -> price * 0.90;

        } else if (choice == 2) {

            rule = price -> price * 0.80;

        } else if (choice == 3) {

            rule = price -> price - 500;

        } else {

            System.out.println("Invalid choice.");
            sc.close();
            return;
        }

        System.out.println("\nPrices after discount:");

        for (double price : prices) {

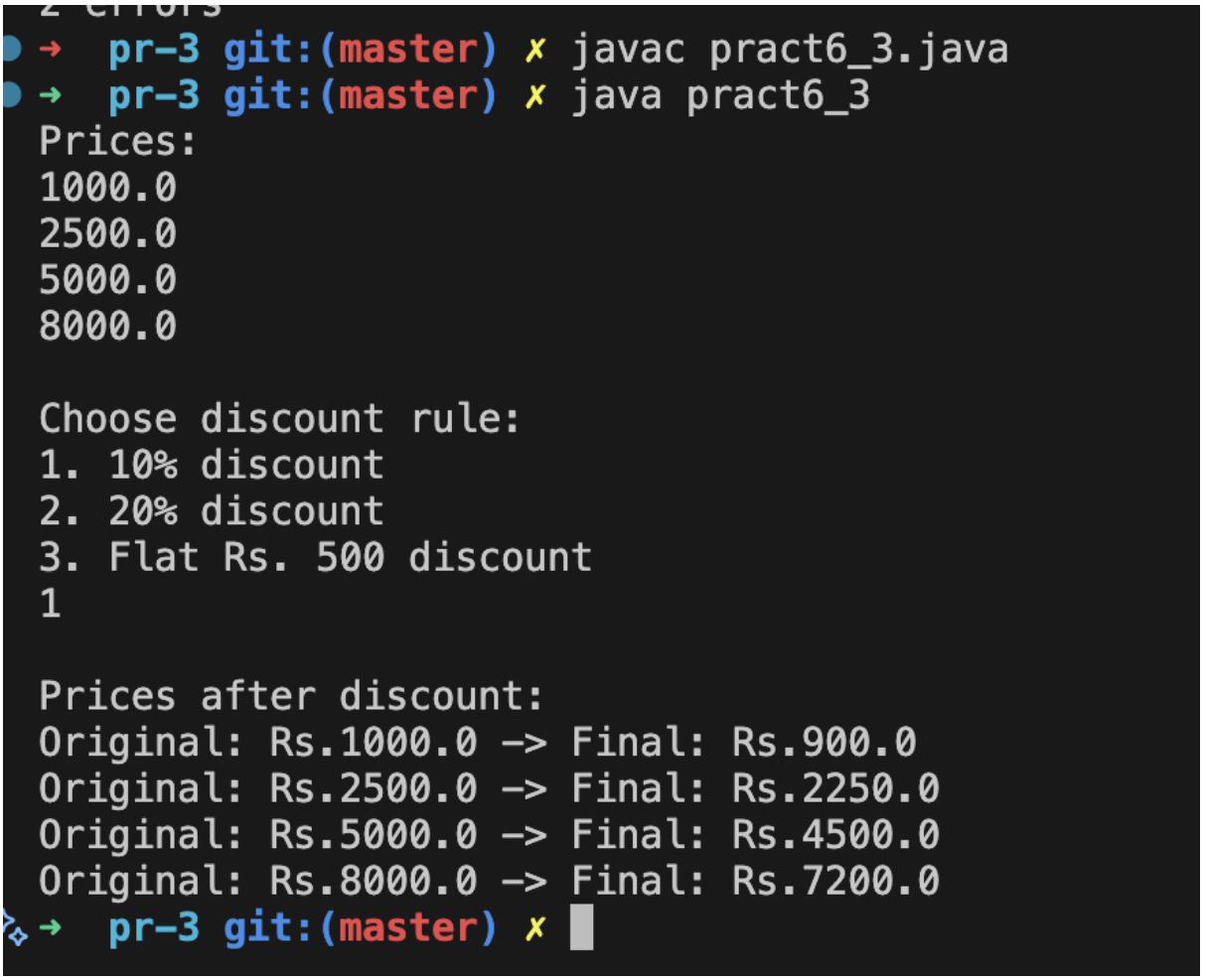
            double finalPrice = rule.apply(price);

            // Prevent negative price
            if (finalPrice < 0) {
                finalPrice = 0;
            }

            System.out.println(
                "Original: Rs." + price +
                " -> Final: Rs." + finalPrice
            );
        }

        sc.close();
    }
}
```

OUTPUT SCREENSHOT :



LEARNING OUTCOME :

- Learned to design an abstract media hierarchy with different media subclasses.
- Applied inheritance, abstraction, and method overriding to implement different late-fee rules.

- Used polymorphism to process different media types through a common Media[] array.
- Learned to calculate individual late fees and the total fee for a returned batch.
- Applied instanceof to identify different media types where required.

Practical – 07

Part A — Practice Programs (1 Hour)

PROBLEM 1	**1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.**

CODE	import java.lang.annotation.*;
	import java.lang.reflect.Field;
	import java.util.*;
	
	@Retention(RetentionPolicy.RUNTIME)
	@Target(ElementType.FIELD)
	@interface NotBlank {
	}
	
	@Retention(RetentionPolicy.RUNTIME)
	@Target(ElementType.FIELD)
	@interface MaxLength {
	  int value();
	}
	
	class SignupForm {
	
	  @NotBlank
	  @MaxLength(10)
	  String username;
	
	  @NotBlank
	  @MaxLength(20)
	  String email;
	
	  SignupForm(String username, String email) {
	  this.username = username;
	  this.email = email;
	  }
	}
	
	public class Main {
	
	  public static void main(String[] args) throws Exception {
	
	  SignupForm form = new SignupForm("",
	"abcdefghijklmnop@gmail.com");
	
	  List<String> errors = new ArrayList<>();

```
        for (Field field : form.getClass().getDeclaredFields()) {

            field.setAccessible(true);

            String value = (String) field.get(form);

            // Check NotBlank
            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null || value.trim().isEmpty()) {
                    errors.add(field.getName() + " cannot be blank");
                }
            }

            // Check MaxLength
            if (field.isAnnotationPresent(MaxLength.class)) {

                MaxLength max = field.getAnnotation(MaxLength.class);

                if (value != null && value.length() > max.value()) {
                    errors.add(field.getName()
                            + " is too long. Maximum "
                            + max.value() + " characters allowed");
                }
            }
        }

        if (errors.isEmpty()) {
            System.out.println("Form is valid");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Main.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Main
username cannot be blank
email is too long. Maximum 20 characters allowed
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7>
```

PROBLEM 2	**2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.**

CODE	import java.lang.annotation.*;
	import java.lang.reflect.Method;
	
	@Retention(RetentionPolicy.RUNTIME)
	@Target(ElementType.METHOD)
	@interface Run {
	}
	
	class MyTests {
	
	  @Run
	  public void test1() {
	  System.out.println("Test 1 executed");
	  }
	
	  @Run
	  public void test2() {
	  System.out.println("Test 2 executed");
	  }
	
	  public void normalMethod() {
	  System.out.println("Normal method");
	  }
	}
	
	public class Mini {
	
	  public static void main(String[] args) throws Exception {
	
	  MyTests obj = new MyTests();
	
	  int count = 0;
	
	  Method[] methods = obj.getClass().getDeclaredMethods();
	
	  for (Method method : methods) {
	
	  if (method.isAnnotationPresent(Run.class)
	  && method.getParameterCount() == 0) {
	
	  method.invoke(obj);

```
            count++;
        }
      }

      System.out.println("Total methods executed: " + count);
   }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Mini.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Mini
Test 2 executed
Test 1 executed
Total methods executed: 2
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7>
```

PROBLEM 3

3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

CODE

```
import java.lang.annotation.*;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
   String name();
}

class Student {

   @Column(name = "id")
   String id;

   @Column(name = "name")
   String name;

   @Column(name = "email")
   String email;
}

public class Columns {

   public static void main(String[] args) throws Exception {

      String[] headers = {"id", "name", "email"};
```

```
        String[] data = {"101", "Om", "om@gmail.com"};

        Student student = new Student();

        Field[] fields = student.getClass().getDeclaredFields();

        for (Field field : fields) {

            if (field.isAnnotationPresent(Column.class)) {

                Column column = field.getAnnotation(Column.class);

                for (int i = 0; i < headers.length; i++) {

                    if (headers[i].equals(column.name())) {

                        field.setAccessible(true);
                        field.set(student, data[i]);

                        break;
                    }
                }
            }
        }

        System.out.println("ID: " + student.id);
        System.out.println("Name: " + student.name);
        System.out.println("Email: " + student.email);
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> javac Columns.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 7> java Columns
ID: 101
Name: Om
Email: om@gmail.com
```

Practical – 08

Part A — Practice Programs (1 Hour)

PROBLEM 1	1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

CODE	import java.util.Scanner;
	
	public class Calculator {
	
	  public static void main(String[] args) {
	  Scanner sc = new Scanner(System.in);
	
	  try {
	  System.out.print("Enter first number: ");
	  double num1 = sc.nextDouble();
	
	  System.out.print("Enter second number: ");
	  double num2 = sc.nextDouble();
	
	  if (num1 < 0 || num2 < 0) {
	  throw new InvalidNumberException("Negative numbers are not allowed!");
	  }
	
	  if(num1==0 || num2==0){
	  throw new InvalidNumberException("Multiplication By Zero is not allowed!");
	  }
	
	  System.out.print("Enter operator (+, -, *, /): ");
	  char op = sc.next().charAt(0);
	
	  System.out.println("You entered: " + num1 + " " + op + " " + num2);
	
	  double result = 0;
	
	  if (op == '+') {
	  result = num1 + num2;
	  } else if (op == '-') {
	  result = num1 - num2;
	  } else if (op == '*') {
	  result = num1 * num2;

```
            } else if (op == '/') {
                if (num2 == 0) {
                    throw new DivideByZeroException("Cannot divide by zero!");
                }
                result = num1 / num2;
            } else {
                throw new InvalidNumberException("Invalid operator entered!");
            }

            System.out.println(num1 + " " + op + " " + num2 + " = " + result);

        } catch (DivideByZeroException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InvalidNumberException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Unexpected error: " + e.getMessage());
        } finally {
            System.out.println("Exiting the program.");
            sc.close();
        }
    }
}

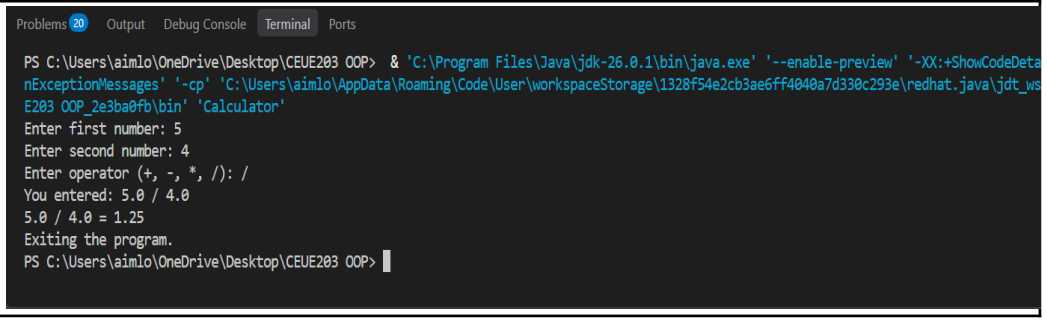
class DivideByZeroException extends Exception {

    public DivideByZeroException(String message) {
        super(message);
    }
}

class InvalidNumberException extends Exception {

    public InvalidNumberException(String message) {
        super(message);
    }
}
```

OUTPUT



PROBLEM 2	2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0.Process a list of requests, catching and reporting each failure without stopping the run.
CODE	

```
import java.util.Scanner;

class outofstockexception extends Exception {

   private int shortage;

   public outofstockexception(int shortage) {
      super("out of stock! shortage: " + shortage);
      this.shortage = shortage;
   }

   public int getshortage() {
      return shortage;
   }
}

class invalidquantityexception extends Exception {

   public invalidquantityexception() {
      super("invalid quantity! must be greater than 0.");
   }
}

class store {

   private int stock;

   public store(int stock) {
      this.stock = stock;
   }

   public void issue(String item, int qty)
         throws outofstockexception, invalidquantityexception {

      if (qty <= 0) {
         throw new invalidquantityexception();
      }

      if (qty > stock) {
         int shortage = qty - stock;
```

```
            throw new outofstockexception(shortage);
        }

        stock -= qty;

        System.out.println("issued: " + qty + " " + item);
        System.out.println("left in stock: " + stock);
    }
}

public class storemanagement {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("enter starting stock: ");
        int stock = sc.nextInt();

        store s = new store(stock);

        System.out.print("enter number of requests: ");
        int requests = sc.nextInt();
        sc.nextLine();

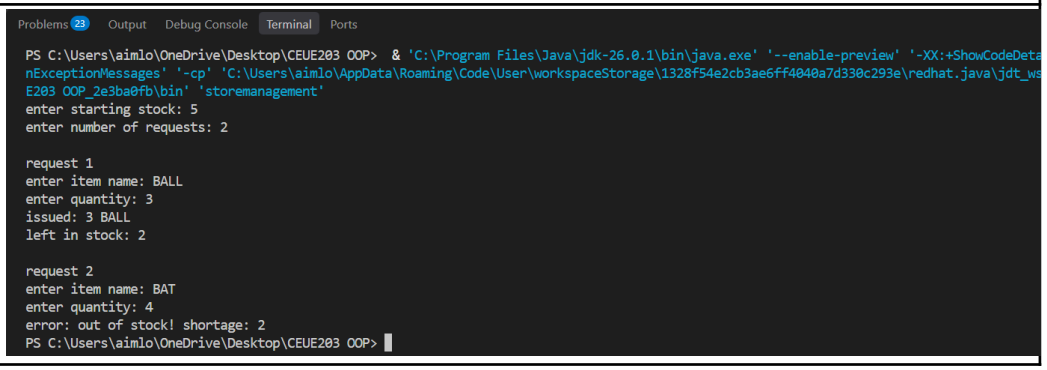
        for (int i = 1; i <= requests; i++) {
            System.out.println("\nrequest " + i);

            System.out.print("enter item name: ");
            String item = sc.nextLine();

            System.out.print("enter quantity: ");
            int qty = sc.nextInt();
            sc.nextLine();

            try {
                s.issue(item, qty);
            } catch (outofstockexception e) {
                System.out.println("error: " + e.getMessage());
            } catch (invalidquantityexception e) {
                System.out.println("error: " + e.getMessage());
            }
        }

        sc.close();
    }
}
```

OUTPUT	

PROBLEM 3	3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with- resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.
CODE	

```
class MyResource implements AutoCloseable {

   void doWork() {
      System.out.println("Resource is doing some work...");
      throw new RuntimeException("Original error occurred");
   }

   @Override
   public void close() {
      System.out.println("Resource is closed");
      throw new RuntimeException("Error while closing resource");
   }
}

public class Main {

   public static void main(String[] args) {

      try (MyResource resource = new MyResource()) {

         System.out.println("Resource is opened");
         resource.doWork();

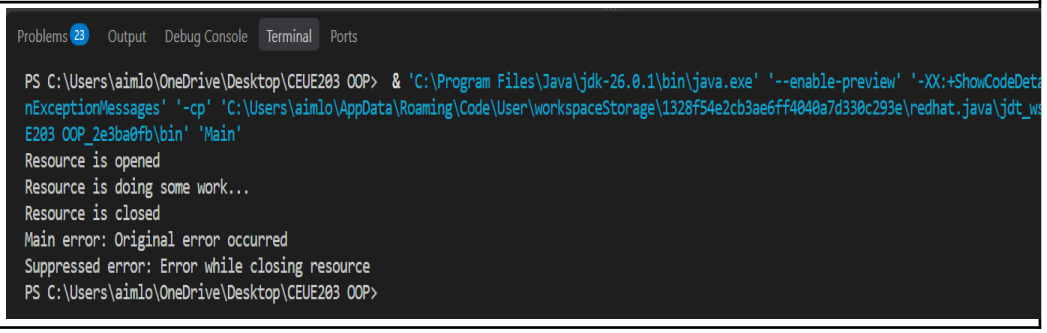
      } catch (Exception e) {

         System.out.println("Main error: " + e.getMessage());

         for (Throwable error : e.getSuppressed()) {
            System.out.println("Suppressed error: " + error.getMessage());
```

```
        }
    }
  }
}
```

OUTPUT


Problems 23
Output
Debug Console
Terminal
Ports
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDeta
nExceptionMessages' '-cp' 'C:\Users\aimlo\AppData\Roaming\Code\User\workspaceStorage\1328f54e2cb3ae6ff4040a7d330c293e\redhat.java\jdt_ws
E203 OOP_2e3ba0fb\bin' 'Main'
Resource is opened
Resource is doing some work...
Resource is closed
Main error: Original error occurred
Suppressed error: Error while closing resource
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP>

Practical - 09

Part A - Practice Programs (1 Hour)

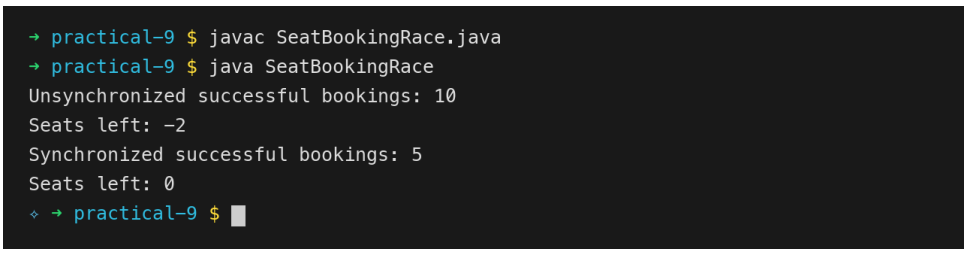
PROBLEM 1	Counter race: Create several threads that increment one shared counter. First run without synchronization and observe that the total may be too low. Then synchronize the increment and confirm the exact total.

CODE	**CounterRace.java**
	`class CounterRace{`
	`static class Counter{`
	`int value=0;`
	`void increment(){value++;}`
	`synchronized void incrementSafely(){value++;}`
	`}`
	`static int runTest(boolean safe)throws InterruptedException{`
	`Counter counter=new Counter();`
	`Thread[]threads=new Thread[10];`
	`for(int i=0;i<threads.length;i++){`
	`threads[i]=new Thread(()->{for(int`
	`j=0;j<100000;j++){if(safe)counter.incrementSafely();else counter.increment();}});`
	`threads[i].start();`
	`}`
	`for(Thread thread:threads)thread.join();`
	`return counter.value;`
	`}`
	`public static void main(String[]args)throws InterruptedException{`
	`System.out.println("Unsynchronized total: "+runTest(false));`
	`System.out.println("Synchronized total: "+runTest(true));`
	`System.out.println("Expected total: 1000000");`
	`}`
	`}`

OUTPUT	`→ practical-9 $ javac CounterRace.java`
	`→ practical-9 $ java CounterRace`
	`Unsynchronized total: 227276`
	`Synchronized total: 1000000`
	`Expected total: 1000000`
	`✧ → practical-9 $`

PROBLEM 2	Seat booking race: A shared counter starts with 5 seats and 10 threads each try to book one seat. Show the race without synchronization, then synchronize book() so no more than 5 bookings succeed.

CODE	**SeatBookingRace.java**
	`import java.util.concurrent.CountDownLatch;`
	`import java.util.concurrent.atomic.AtomicInteger;`
	`class SeatBookingRace{`
	`static class TicketOffice{`
	`int seatsLeft=5;`
	`boolean`
	`bookUnsafe(){if(seatsLeft>0){try{Thread.sleep(5);}catch(InterruptedException`
	`e){Thread.currentThread().interrupt();return false;}seatsLeft--;return`
	`true;}return false;}`
	`synchronized boolean bookSafely(){if(seatsLeft>0){seatsLeft--;return true;}return`
	`false;}`
	`}`
	`static void runTest(boolean safe)throws InterruptedException{`
	`TicketOffice office=new TicketOffice();`
	`AtomicInteger booked=new AtomicInteger();`
	`CountDownLatch ready=new CountDownLatch(10);`
	`CountDownLatch start=new CountDownLatch(1);`
	`Thread[]threads=new Thread[10];`
	`for(int i=0;i<threads.length;i++){`
	`threads[i]=new`
	`Thread(()->{ready.countDown();try{start.await();}catch(InterruptedException`
	`e){Thread.currentThread().interrupt();return;}boolean`
	`success=safe?office.bookSafely():office.bookUnsafe();if(success)booked.incrementAn`
	`dGet();});`
	`threads[i].start();`
	`}`
	`ready.await();start.countDown();`
	`for(Thread thread:threads)thread.join();`
	`System.out.println((safe?"Synchronized":"Unsynchronized")+" successful bookings:`
	`"+booked.get());`
	`System.out.println("Seats left: "+office.seatsLeft);`
	`}`
	`public static void main(String[]args)throws InterruptedException{`
	`runTest(false);`
	`runTest(true);`
	`}`
	`}`

OUTPUT	

PROBLEM 3	(For Advanced Learners - home) Split a large array across several threads. First let all threads add to one shared total without synchronization and observe the wrong result. Fix it using (1) a synchronized addition and (2) a thread-local partial total; compare the methods.

CODE	**ParallelArraySum.java**
	`class ParallelArraySum{`
	`static class SharedTotal{`
	`long value=0;`
	`void addUnsafe(long amount){value+=amount;}`
	`synchronized void addSafely(long amount){value+=amount;}`
	`}`
	`static long sumWithSharedTotal(long[]data,int threadCount,boolean safe)throws`
	`InterruptedException{`
	`SharedTotal total=new SharedTotal();`
	`Thread[]threads=new Thread[threadCount];`
	`for(int t=0;t<threadCount;t++){`
	`final int start=t*data.length/threadCount;`
	`final int end=(t+1)*data.length/threadCount;`
	`threads[t]=new Thread(()->{for(int`
	`i=start;i<end;i++){if(safe)total.addSafely(data[i]);else`
	`total.addUnsafe(data[i]);}});`
	`threads[t].start();`
	`}`
	`for(Thread thread:threads)thread.join();`
	`return total.value;`
	`}`
	`static long sumWithLocalTotals(long[]data,int threadCount)throws`
	`InterruptedException{`
	`long[]partial=new long[threadCount];`
	`Thread[]threads=new Thread[threadCount];`
	`for(int t=0;t<threadCount;t++){`
	`final int id=t;`
	`final int start=t*data.length/threadCount;`
	`final int end=(t+1)*data.length/threadCount;`
	`threads[t]=new Thread(()->{long subtotal=0;for(int`
	`i=start;i<end;i++)subtotal+=data[i];partial[id]=subtotal;});`
	`threads[t].start();`
	`}`
	`for(Thread thread:threads)thread.join();`
	`long total=0;for(long subtotal:partial)total+=subtotal;return total;`
	`}`
	`public static void main(String[]args)throws InterruptedException{`
	`int size=1000000;int threadCount=8;`
	`long[]data=new long[size];`
	`for(int i=0;i<size;i++)data[i]=1;`
	`System.out.println("Expected total: "+size);`
	`System.out.println("Unsynchronized total:`
	`"+sumWithSharedTotal(data,threadCount,false));`
	`System.out.println("Synchronized total:`
	`"+sumWithSharedTotal(data,threadCount,true));`
	`System.out.println("Thread-local total: "+sumWithLocalTotals(data,threadCount));`
	`}`
	`}`

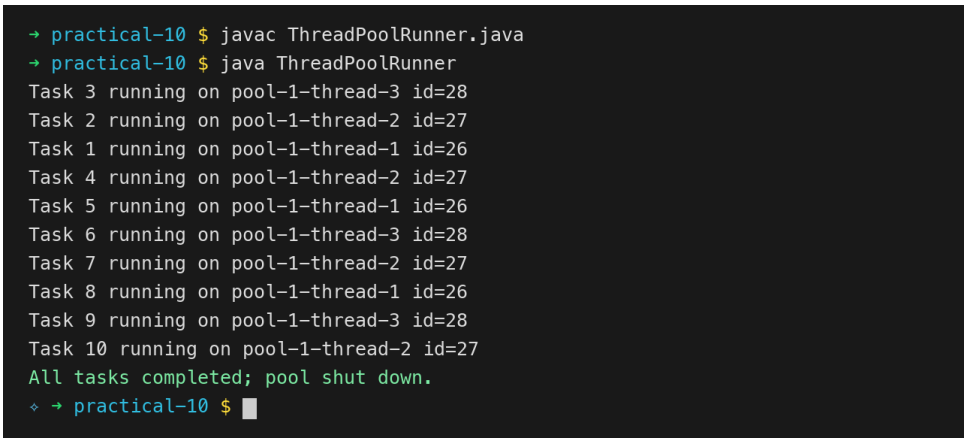
OUTPUT	

Practical - 10

Part A - Practice Programs (1 Hour)

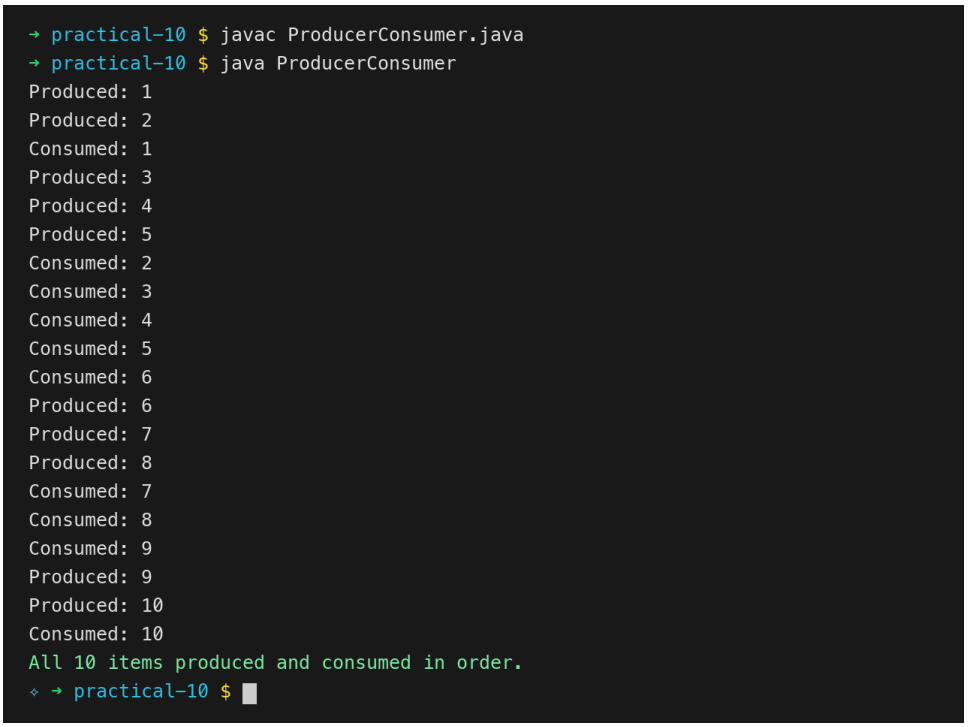
PROBLEM 1	Thread-pool runner: Create a fixed pool of 3 threads and submit 10 Runnable tasks. Each task prints its thread name and ID, sleeps briefly, and the program shuts down the pool and waits for completion. Observe that the pool threads are reused.

CODE	**ThreadPoolRunner.java**
	`import java.util.concurrent.ExecutorService;`
	`import java.util.concurrent.Executors;`
	`import java.util.concurrent.TimeUnit;`
	`class ThreadPoolRunner{`
	`public static void main(String[]args)throws InterruptedException{`
	`ExecutorService pool=Executors.newFixedThreadPool(3);`
	`for(int i=1;i<=10;i++){`
	`final int taskId=i;`
	`pool.execute(()->{System.out.println("Task "+taskId+" running on "+Thread.currentThread().getName()+" id="+Thread.currentThread().getId());try{Thread.sleep(80);}catch(InterruptedException e){Thread.currentThread().interrupt();}});`
	`}`
	`pool.shutdown();`
	`if(pool.awaitTermination(1,TimeUnit.MINUTES))System.out.println("All tasks completed; pool shut down.");`
	`}`
	`}`

OUTPUT	

PROBLEM 2	Producer-consumer: One thread adds numbers to a small shared buffer and another removes them. Coordinate the threads with wait() and notifyAll(). Produce and consume a fixed number of items in order, with none lost.

CODE	**ProducerConsumer.java**
	`class ProducerConsumer{`
	`static class Buffer{`
	`private final int[]items=new int[3];`
	`private int head=0,tail=0,count=0;`
	`synchronized void put(int value)throws InterruptedException{`
	`while(count==items.length)wait();`
	`items[tail]=value;tail=(tail+1)%items.length;count++;`
	`notifyAll();`
	`}`
	`synchronized int take()throws InterruptedException{`
	`while(count==0)wait();`
	`int value=items[head];head=(head+1)%items.length;count--;`
	`notifyAll();`
	`return value;`
	`}`
	`}`
	`public static void main(String[]args)throws InterruptedException{`
	`Buffer buffer=new Buffer();int total=10;`
	`Thread producer=new Thread(()->{try{for(int`
	`i=1;i<=total;i++){buffer.put(i);System.out.println("Produced:`
	`"+i);}}catch(InterruptedException`
	`e){Thread.currentThread().interrupt();}},"Producer");`
	`Thread consumer=new Thread(()->{try{for(int`
	`i=1;i<=total;i++)System.out.println("Consumed:`
	`"+buffer.take());}catch(InterruptedException`
	`e){Thread.currentThread().interrupt();}},"Consumer");`
	`producer.start();consumer.start();producer.join();consumer.join();`
	`System.out.println("All "+total+" items produced and consumed in order.");`
	`}`
	`}`

OUTPUT	

PROBLEM 3	(For Advanced Learners - home) Reproduce a deadlock between two threads that lock two objects in opposite order. Detect the deadlock, then prevent it in a two-account transfer example by always locking the lower account number first.

CODE	**DeadlockDemo.java**
	`import java.lang.management.ManagementFactory;`
	`import java.lang.management.ThreadMXBean;`
	`import java.util.concurrent.CountDownLatch;`
	`class DeadlockDemo{`
	`static final Object LOCK_A=new Object(),LOCK_B=new Object();`
	`static final CountDownLatch READY=new CountDownLatch(2);`
	`static void lockOpposite(Object first,Object`
	`second){synchronized(first){READY.countDown();try{READY.await();}catch(Interrupted`
	`Exception e){Thread.currentThread().interrupt();return;}synchronized(second){}}}`
	`static boolean detectDeadlock()throws InterruptedException{`
	`ThreadMXBean bean=ManagementFactory.getThreadMXBean();`
	`for(int`
	`i=0;i<30;i++){long[]ids=bean.findDeadlockedThreads();if(ids!=null&&ids.length>=2)r`
	`eturn true;Thread.sleep(100);}`
	`return false;`
	`}`
	`static class Account{`
	`final int number;int balance;`
	`Account(int number,int balance){this.number=number;this.balance=balance;}`
	`}`
	`static void transfer(Account from,Account to,int amount){`
	`Account first=from.number<to.number?from:to;`
	`Account second=from.number<to.number?to:from;`
	`synchronized(first){synchronized(second){from.balance-=amount;to.balance+=amount;}`
	`}`
	`}`
	`public static void main(String[]args)throws InterruptedException{`
	`Thread one=new Thread(()->lockOpposite(LOCK_A,LOCK_B),"Lock-A-then-B");`
	`Thread two=new Thread(()->lockOpposite(LOCK_B,LOCK_A),"Lock-B-then-A");`
	`one.setDaemon(true);two.setDaemon(true);one.start();two.start();`
	`System.out.println("Deadlock detected: "+detectDeadlock());`
	`Account a=new Account(1,100),b=new Account(2,100);`
	`Thread t1=new Thread(()->transfer(a,b,10));`
	`Thread t2=new Thread(()->transfer(b,a,10));`
	`t1.start();t2.start();t1.join();t2.join();`
	`System.out.println("Ordered-lock transfers completed.");`
	`System.out.println("Account 1 balance: "+a.balance);`
	`System.out.println("Account 2 balance: "+b.balance);`
	`}`
	`}`

OUTPUT	`→ practical-10 $ javac DeadlockDemo.java`
	`→ practical-10 $ java DeadlockDemo`
	`Deadlock detected: true`
	`Ordered-lock transfers completed.`
	`Account 1 balance: 100`
	`Account 2 balance: 100`
	`→ practical-10 $`